



THE UNIVERSITY OF
SYDNEY

Room Number _____

Seat Number _____

Student Number

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue

Computer Science

EXAMINATION

Practice Exam I

COMP9001 Introduction to Programming

EXAM WRITING TIME: 2 hours

READING TIME: 10 minutes

EXAM CONDITIONS:

This is a RESTRICTED OPEN book exam - specified materials permitted

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Calculator – non-programmable

Formula sheet (provided in the exam paper by unit coordinator)

MATERIALS TO BE SUPPLIED TO STUDENTS:

Blank scratch paper (student will be provided with answer booklet)

INSTRUCTIONS TO STUDENTS:

Do not turn over this page until you are told!

- Reading time is strictly *no writing*
- Please read all the questions first before you begin writing
- Answer all questions in the spaces provided in this question booklet. Do not write your answers in the answer booklets, which are only to be used for drafting and will not be graded.
- Return this question booklet and any additional answer booklets or materials.
- Write your final answers in black or blue ink, not pencil. Take care to write clearly and legibly. Drawing diagrams using pencil is permitted.
- Please write your student number in the space provided at the top of every odd numbered page of this question booklet.
- State clearly in spaces below the question if the answer is continued on the extra working pages
- Do not remove any tear or remove any pages from the booklet. Staff are not responsible for missing pages in this case. If you find there are missing pages, please alert the examination supervisors.

Please tick the box to confirm that your examination paper is complete.

☐

Question 1 (20 marks):

/20

1.a:

/2

Mark the following statements true or false:

- (a) ☐ while loops will be endless without a break statement
- (b) ☐ a variable value can change over the duration of a program
- (c) ☐ the code `x = 3` is a comparison of x with the value of 3
- (d) ☐ python does not compile code

1.b:

/2

Mark the following statements true or false:

- (a) ☐ `g(f(x))`. The function `f()` is evaluated first
- (b) ☐ tuples are immutable in python
- (c) ☐ a dictionary is a collection that cannot contain duplicate values
- (d) ☐ a stack will count the number of times a loop has executed

1.c:

/1

What is the value of m after `m = int(9/2)` is executed?

Put your answer here:

1.d:

/1

What is the value of b after the following statements are executed?

```
1 p = (25 + 5) > 30 and (50 - 20 == 30)
2 b = (not p) and (20 < 21)
```

Put your answer here:

COMP9001; Write your SID:

1.e:

/2

Create a variable and initialise it to a list with 4 numbers, 10, 20, 30 and 40:

1.f:

/2

What is the output after the following statements are executed?

```
1 if (18 - 9) > 7:  
2     print ("Msg 3")  
3 elif (18 - 9) > 2:  
4     print ("Msg 5")  
5 print ("Msg 2")
```

Put your answer here:

1.g:

/2

When would you use logical operators in an `if` statement? Give an example

1.h:

/2

Create a variable and initialise it as a tuple with 4 objects, 20, 'Jim', None and 30.0

1.i:

/4

Given the following loop, draw a flowchart / control flow diagram

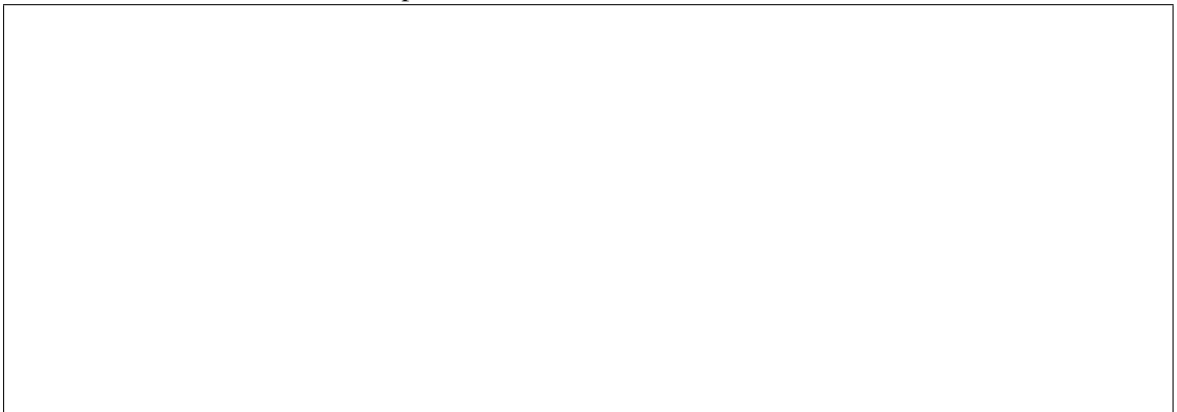
```
1 i = 1
2 while i < 9:
3     i = i + 10
4     i = i / 2
5 print("Tony says " + str(i))
```



1.j:

/2

Draw the truth table for the and operator



Question 2 (20 marks):

/20

2.a:

/6

We require a program to read all integers from the command line and print the minimum value.

The program will be run like this:

```
python print_min.py 34 1 25 -2 0 60 37 36 89 -43 18 80 49 19 -50
```

The following code is the entire contents of `print_min.py`. Find as many *different* errors (of any kind, syntax, or logical) as you can. Assume that `k` are intended to be the list of command line arguments. For each error state what line(s) it is on and why it is an error. *Putting just the line number with no explanation will get no mark.*

```
1 def output_min(k)
2     a = len(k)
3
4     for i in range(10)
5         i += 1
6         if k[i] > min_v:
7             min_v = k[i]
8         else
9             min_v = a
```

There are at least 8 errors, you need to identify 6 for full marks.

(a) line: ; error is:

(b) line: ; error is:

(c) line: ; error is:

(d) line: ; error is:

(e) line: ; error is:

(f) line: ; error is:

(g) line: ; error is:

(h) line: ; error is:

2.b:

/2

What could possibly go wrong with the following code when it is executed? Describe two of the errors and explain what kind of error each will cause. Use line numbers in your answer.

```
1 import sys
2
3 def so_wrong(contents):
4     x = int(contents[1])
5     new_contents = []*x
6     i = 0
7     while i < len(contents):
8         new_contents[i] = 1.0 / (5 - i)
9         i += 1
10
11 so_wrong(sys.argv)
```

i

ii

2.c:

/2

Mark the following statements true or false:

- (a) ☐ The `error` keyword is used to create a new exception
- (b) ☐ Syntax errors are automatically detected by the compiler
- (c) ☐ Exceptions will always crash your program, even if they are caught
- (d) ☐ Opening a file for reading will always be a successful

2.d:

/10

Write a complete program called `print_even_lines` in the space below with the following behaviour:

- It opens a text file with the filename given as the second command-line argument;
- It prints every even line of the input file (first line is considered odd);
- Any other arguments are ignored;
- On any error, the message `"error"` is printed to the console and the program will terminate.

For example, to run the program with the file `hello.txt`:

```
python print_even_lines.py hello.txt
```

Make sure you handle and report *any possible exceptions* that might arise.

Question 3 (15 marks):

/15

3.a:

/7

Write a module called `Car` with three functions and two global variables. Global variables `colour` is type `str`, and `top_speed` is type `float`. Their initial values are `"none"` and `-1.337`

Function `initialise(p_colour, p_top_speed)` sets the global variable. Each parameter should be type checked separately. If the type of a parameter is incorrect, then your code should verify and set each of these to their default value. If parameter `p_colour` is not type `str`, set `colour` $\rightarrow$ `"white"`. If parameter `p_top_speed` is not type `float`, set `top_speed` $\rightarrow$ `120`. Function `is_fast()` returns `True` if global variable `top_speed` is greater than or equal to `200`, otherwise `False`.

Function `is_super_fast()` returns `True` if the global variable `colour` is `red` and also has global variable `top_speed` value greater than or equal to `200`, otherwise `False`.

Hint: use `isinstance` for type checking

3.b:

/8

Write a program called `divisibles` that takes n strings from command line arguments. A valid divisible includes a forward slash symbol `/` in between two numbers (numerator and denominator). For each valid divisible, your program should output the calculated floating point number of division. A string is not a divisible, and should be ignored when: 1) the string does not contain exactly one forward slash, or 2) the denominator or numerator could not be converted to a number, or 3) the division is unsuccessful.

Hint: use `sys` module, string method `split`, and exception handling

Examples

```
python divisibles 1/2 abc n/m 3.0/2 6/3 3/3/1
0.5 1.5 2
```

```
python divisibles 2/4 abc n/m 10/2 p/2 3/0 "a 4/2 b"
0.5 5.0
```

Continue your answer here

COMP9001; Write your SID:

Notes and reminders

Common types: `int, float, bool, str`

Common escape characters: `\n \t \' \" \\`

List terms: `[ ]`

Type information: `type(), dir() isinstance(object, class)`

Parsing methods: `int(), float(), str()`

Collection classes: `list, dict, set, tuple`

Collection methods and functions: `append, insert, remove, pop, index, reverse, copy`

Keywords

`def` – Keyword for defining a function

`yield` – Returns a value from a generator object

`import` – Imports a module for your program

`raise` – Raises an exception

`break, continue, return, assert`

Common Modules

`sys` – module, provides access to command line arguments.

`os.path` – module, provides access to checking file paths.

iterable collection operations:

`sorted()` – sort the iterable collection

`reversed()` – reversed the iterable collection

`list(iterable)` – returns a copy of list whose items are the same and in the same order as iterable items

`len` – returns the length of the iterable collection

`next` – returns the next item in the iterator, otherwise raises `StopIteration`

`iter` – returns an iterator using the object passed

Control Flow Syntax:

- `try: ... except [...as ...]: ...[ finally: ...]`
- `for ... in ... : ... [else: ...]`
- `while ... : ... [else: ...]`
- `if ... : ... [elif ... : ...] [else: ...]`

Common Exceptions: `Exception, EOFError, ZeroDivisionError, SyntaxError, AssertionError, KeyError, FileNotFoundError, IndexError, NameError.`

Arithmetic Operators: + - * / // **

Logical Operators: not and or == != >= <= < >

Bitwise Operators: & | ^ << >>

Assignment Operators: = += *= -= /=

Useful functions, methods and constructors:

Input/Output functions:

`input()` – Allows for reading from standard input

`open(filepath, mode)` – Opens a file for reading, writing or append.

File access mode 'r' – Read mode 'w' – Writing mode 'a' – Append mode

File methods:

`close()` – Closes the file

`read(n)` – Reads n characters from the file

`readline()` – Reads all input up until a new line

`readlines()` – Reads all lines from the file

`flush()` – Writes data stored in the buffer to the file immediately

`write()` – Writes data to a file

`print()` – Write the string representation of the object(s) to standard out

String methods (`str`):

`==` – Check if this string is equal to the other string

`s[i]` – Gets a character from `s` at index `i`

`split` – Splits a string based on a delimiter, returns a list of strings

`lower` – Returns a string in lowercase

`upper` – Returns a string in uppercase

`strip` – Returns a string after characters matching a pattern are eliminated from the start and end of the string.

`format` – Returns a formatted version of the string. Using substitution from mapping.

Substitutions are identified by braces { and }